

Relatório do Projeto de Banco de Dados

Título	Web Chat Room Based Application
Descrição	Simples aplicação de Web Chat em tempo real baseado em salas. O sistema contará com UserAuthLess e utilizando triggers, procedure e <i>notify</i> para encaminhar novas mensagens a sala. Deste modo, utilizando Event Listeners será possível sincronizar as mensagens entre todos os usuários da sala.
Membros	• Ana Paula Teixeira • Jaine De Senna Santos • Marcelo Victor Melo Nunes • Maria Das Graças Dias Amorim • Zaú Júlio Araújo Galvão
SGBD	PostgreSQL
Instalação do Ambiente	Docker&Postgres - Tutorial
Documento	Google Docs - Relatório do Projeto...

Sumário

1.	Descrição Prática	2
2.	Requisitos	3
3.	Modelagem	4
4.	SGBD	6
5.	DDL: A Criação	7
6.	Conceitos iniciais	9
7.	Instalação do Ambiente	10
8.	Referências	11

1. Descrição Prática

O sistema baseia-se na transmissão de mensagens de um cliente para um grupo em “tempo real”, logo pode ser utilizado uma abordagem com FUNCTION’s (*CREATE FUNCTION*, n.d.) e NOTIFY/Listen (*NOTIFY*, n.d.) para interceptar a inserção de uma nova instância no banco de dados e “notificar” outro processo da ocorrência. Para realizar a invocação de uma função, mediante determinado evento, utilizamos “*trigger*” (*Trigger Procedures*, n.d.), (*CREATE TRIGGER*, n.d.), conhecido como gatilhos, eles fornecem a chamada para a FUNCTION criada anteriormente e por fim, executa o NOTIFY para o cliente definido. Agora um exemplo simples do funcionamento do sistema:

1. Um novo usuário entra no sistema após alguma verificação.
2. O “a” usuário cria uma sala.
3. Outro usuário “b” entra na mesma sala.
4. O usuário “b” envia uma mensagem na sala.
 - a. O Postgres chama o “*trigger*” criado para ser executado antes da inserção no banco de dados.
 - b. “*Trigger*” chama a função que propriamente realiza a inserção.
 - c. Ao fim da execução da função, NOTIFY é chamado.
 - d. O NOTIFY envia uma notificação via socket a um cliente, indicando que houve alterações no banco de dados, podendo ser somente a própria alteração como notificação.
 - i. Esse cliente é responsável por notificar os outros membros da sala (nesse caso o usuário “a”), para que eles possam atualizar suas interfaces com os demais usuários.
 - ii. Este cliente pode enviar somente um aviso de mudanças no banco de dados, ou, enviar a modificação em si, evitando com que sejam necessárias mais requisições ao SGBD.
 - iii. Contudo, é possível que ocorram várias alterações “simultâneas” em alguns cenários, neste caso é possível que ocorram problemas decorrentes da sincronização de múltiplas instâncias. Já são conhecidas inúmeras soluções para estes problemas de concorrência.
5. Por fim, o usuário “a”, e todos os possíveis outros membros da sala, recebem a mensagem.

Prosseguindo com a idealização do sistema, propomos o seguinte modelo baseado em quatro entidades: **Usuário**, com os seguintes atributos (identificador, nome, email e situação de banimento da sala). **Sala**, com (identificador, nome, data de criação, data de fechamento e o identificador de quem foi criada). **Mensagem**, contendo (identificador, conteúdo, data de envio, o identificador de quem a enviou e a sala a qual pertence. Por fim, **Membro**, a entidade que fornece o relacionamento entre a sala e o usuário.

2. Requisitos

A tabela a seguir contém os requisitos funcionais do projeto, idealizados a partir dos dados,

Requisito	Descrição	Ator
RF001 - Criar uma sala	Um usuário cria uma sala para receber novos usuários/membros. Uma sala tem código, nome, data de abertura e fechamento.	Usuário
RF002 - Fechar uma sala	O usuário criador da sala, pode fechá-la, expulsando todos os membros da sala.	Membro
RF003 - Criar um usuário	Um usuário pode criar salas e participar de outras salas tornando-se membro(caso não esteja banido da sala). Um usuário tem código, nome, email e status de banimento.	Usuário
RF004 - Retornar usuário	O usuário pode solicitar informações como status de banimento de salas, seu próprio nome e email.	Usuário
RF005 - Alterar um usuário	Um usuário pode alterar seus próprios dados, como nome e email.	Usuário
RF006 - Excluir usuário	O usuário pode solicitar exclusão do próprio perfil.	Usuário
RF007 - Criar membro	O usuário torna-se membro ao entrar em uma sala. Contém as informações do código da sala e do usuário.	Membro
RF008 - Excluir Membro	O usuário sai da sala e deixa de ser membro daquela instância.	Membro
RF009 - Banir usuário	O membro criador da sala pode expulsar outro usuário presente na sala.	Membro
RF010 - Postar mensagem	Um membro pode postar uma nova mensagem na sala. Uma mensagem contém código, conteúdo, código de usuário, código da sala e data de envio.	Membro
RF011 - Excluir mensagem	Um membro pode solicitar a exclusão de uma mensagem enviada por ele para a sala. O membro criador da sala também pode solicitar a exclusão de uma mensagem de outro membro.	Membro

3. Modelagem

Para implementar a lógica de operação básica deste sistema, começemos idealizando a estrutura de entidades e seus relacionamentos a partir do modelo a seguir:

Modelo entidade-relacionamento proposto, com a ilustração da idealização na **Figura 1.**:

Ex.: **table**(*pk*, attr, attr, fk);

- **user**(*id*, name, email, banned);
- **room**(*id*, name, createdIn, closedIn, createBy);
- **message**(*id*, content, sendeIn, userId, roomId);
- **member**(*id*, userId, roomId);

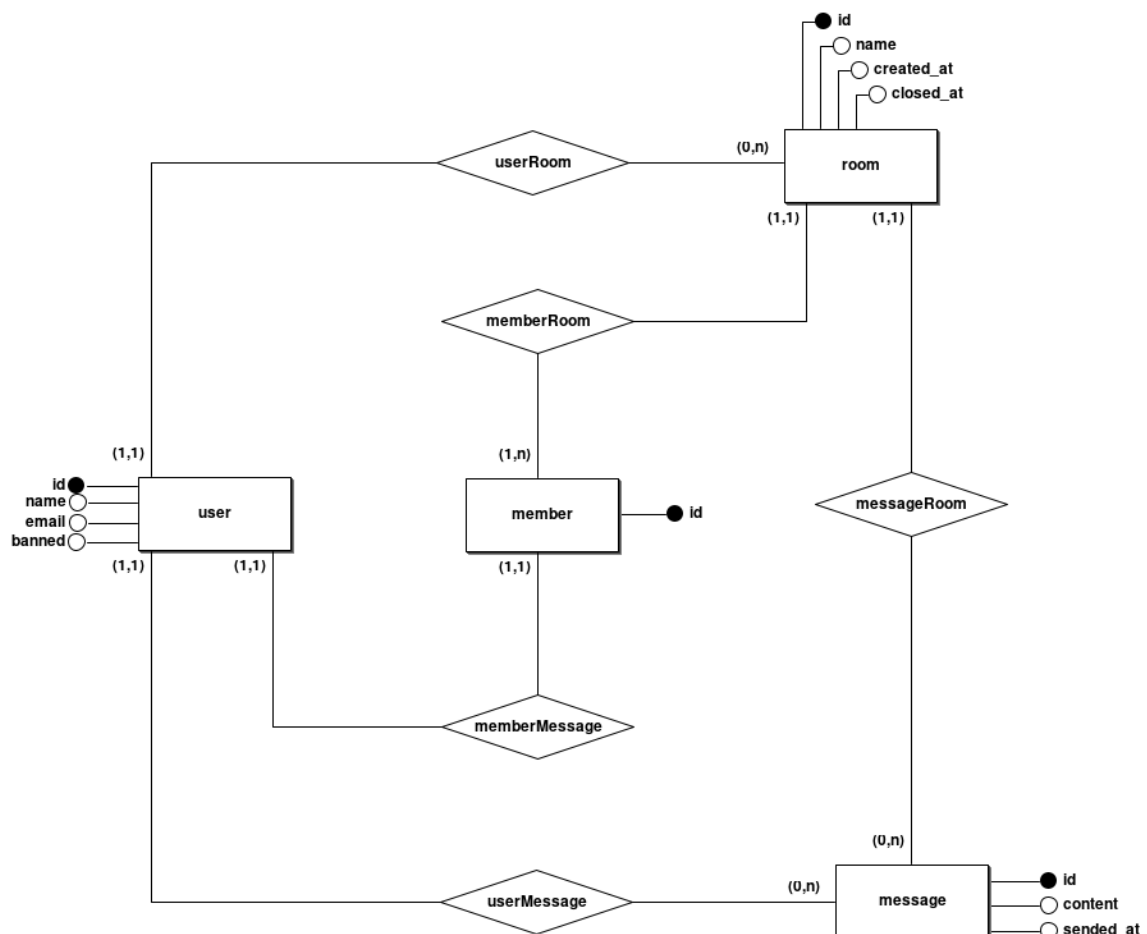


Figura 1. Implementação gráfica do modelo conceitual proposto.

A figura acima, ilustra de forma convencional a modelagem entidade-relacionamento do sistema, ela fornece uma visualização abstrata da arquitetura de dados do sistema. A figura seguinte ilustra a idealização de um modelo lógico implementável em acordo com o modelo conceitual acima.

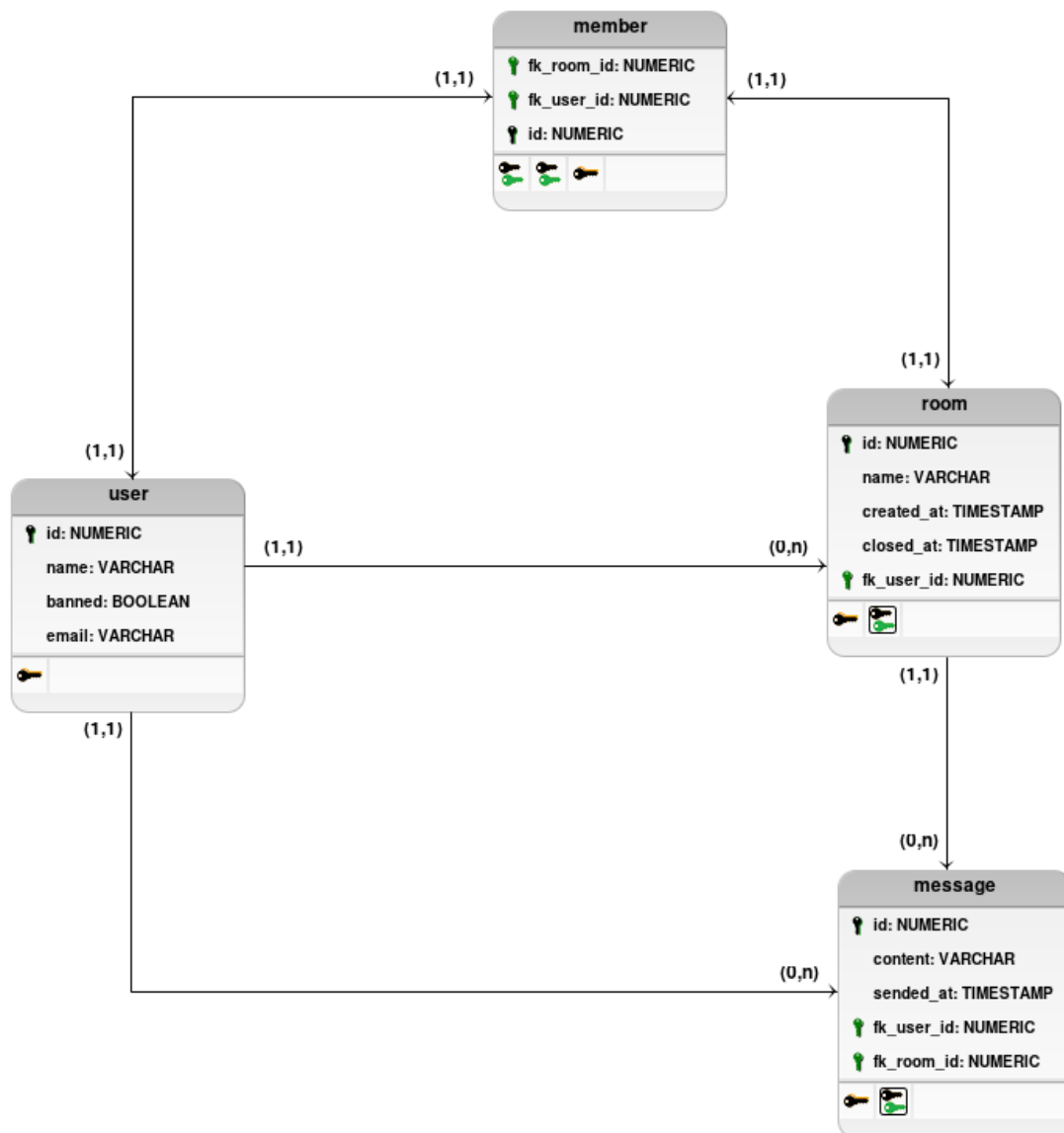


Figura 2. Conversão do modelo conceitual para o modelo lógico.

O modelo lógico visa fornecer uma visão técnica do esquema de tabelas do banco de dados do sistema.

4. SGBD

O PostgreSQL é um sistema de gerenciamento de bancos de dados objeto-relacional de uso geral que usa a linguagem SQL e é um dos mais avançados sistema de banco de dados de código aberto. Foi desenvolvido com base no POSTGRES 4.2 do Berkeley Computer Science Department, da Universidade da Califórnia.

O Postgres foi projetado para rodar em plataformas semelhantes ao UNIX. No entanto, também foi projetado para ser portátil, para que pudesse ser executado em várias plataformas, como Mac OS X, Solaris e Windows.

Este SGBD conquistou uma forte reputação por sua arquitetura comprovada, confiabilidade, integridade de dados, conjunto de recursos robustos, extensibilidade e dedicação da comunidade de código aberto por trás do software para fornecer soluções inovadoras e de alto desempenho de maneira consistente. PostgreSQL é executado em todos os principais sistemas operacionais, é compatível com ACID desde 2001 e possui complementos poderosos, como o popular extensor de banco de dados geoespacial PostGIS.

5. DDL: A Criação

Repositório: [GitHub/Web Chat Room Postgres/create.sql](https://github.com/Postgres/create.sql)

```
CREATE TABLE room(  
  id NUMERIC PRIMARY KEY NOT NULL,  
  name VARCHAR [16] NOT NULL,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  closed_at TIMESTAMP DEFAULT NULL,  
  fk_user_id NUMERIC NOT NULL  
);
```

```
CREATE TABLE message(  
  id NUMERIC PRIMARY KEY NOT NULL,  
  content VARCHAR [254] NOT NULL,  
  sented_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  fk_user_id NUMERIC NOT NULL,  
  fk_room_id NUMERIC NOT NULL  
);
```

```
CREATE TABLE userr(  
  id NUMERIC PRIMARY KEY NOT NULL,  
  name VARCHAR [16] NOT NULL,  
  banned BOOLEAN NOT NULL DEFAULT FALSE,  
  email VARCHAR NOT NULL  
);
```

```
CREATE TABLE member(  
  id NUMERIC PRIMARY KEY NOT NULL,  
  fk_user_id NUMERIC NOT NULL,  
  fk_room_id NUMERIC NOT NULL  
);
```

```
ALTER TABLE  
  room  
ADD CONSTRAINT FK_room_user FOREIGN KEY (fk_user_id) REFERENCES  
userr(id);
```

```
ALTER TABLE  
  message  
ADD CONSTRAINT FK_message_user FOREIGN KEY (fk_user_id) REFERENCES  
userr(id);
```

ALTER TABLE

message

ADD CONSTRAINT FK_message_room FOREIGN KEY (fk_room_id) REFERENCES
room(id);

ALTER TABLE

member

ADD CONSTRAINT FK_member_room FOREIGN KEY (fk_room_id) REFERENCES
room(id) ON DELETE RESTRICT;

ALTER TABLE

member

ADD CONSTRAINT FK_member_userr FOREIGN KEY (fk_user_id) REFERENCES
userr(id) ON DELETE RESTRICT;

6. Conceitos iniciais

7. Instalação do Ambiente

8. Referências

CREATE FUNCTION — define a new function. (n.d.). Retrieved 08 02, 2021, from <https://www.postgresql.org/docs/current/sql-createfunction.html>

CREATE TRIGGER. (n.d.). Retrieved 08 02, 2021, from <https://www.postgresql.org/docs/9.1/sql-createtrigger.html>

Dimitri Fontaine. (2018). PostgreSQL LISTEN/NOTIFY. Retrieved 08 02, 2021, from <https://tapoueh.org/blog/2018/07/postgresql-listen-notify/>

NOTIFY. (n.d.). **NOTIFY.** Retrieved 08 02, 2021, from <https://www.postgresql.org/docs/9.0/sql-notify.html>

Trigger Procedures. (n.d.). Retrieved 08 02, 2021, from <https://www.postgresql.org/docs/9.2/plpgsql-trigger.html>